

LAPORAN TUGAS KECIL 3

IF2211 STRATEGI ALGORITMA

**Ice Sliding Puzzle Solver dengan Algoritma UCS, Greedy Best
First Search, dan A***

Disusun oleh:

Yusuf Faishal Listyardi

NIM: 13524014

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

2026

Daftar Isi

1	Pendahuluan	2
1.1	Deskripsi Singkat Masalah	2
1.2	Ruang Lingkup Implementasi	3
2	Algoritma dan Implementasi	4
2.1	Representasi Papan dan State	4
2.2	Simulasi Sliding	4
2.3	Uniform Cost Search	5
2.4	Greedy Best First Search	6
2.5	A* Search	7
2.6	Perbandingan Algoritma	8
2.7	Heuristik	10
2.7.1	H1: Manhattan ke Goal	10
2.7.2	H2: Manhattan ke Target Wajib Berikutnya	11
2.7.3	H3: Rantai Manhattan Checkpoint	11
2.8	Admissibility Heuristik	12
2.9	Bonus GUI JavaFX	14
3	Struktur Program dan Cara Menjalankan	15
3.1	Struktur Direktori	15
3.2	Cara Menjalankan	16
4	Eksperimen dan Hasil Pengujian	17
4.1	Test Case 1	17
4.2	Test Case 2	18
4.3	Test Case 3	19
4.4	Test Case 4	21
4.5	Test Case 5	22
5	Penutup	23
5.1	Kesimpulan	23
5.2	Saran	23
5.3	Repository	23
5.4	Checklist Pemenuhan Spesifikasi	24
5.5	Pernyataan tidak melakukan kecurangan	24

BAB 1

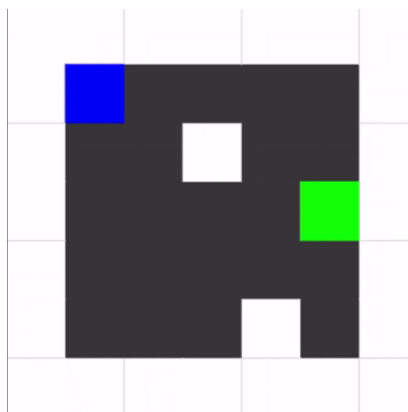
Pendahuluan

1.1 Deskripsi Singkat Masalah

Ice Sliding Puzzle adalah permainan pencarian jalur pada papan berukuran $N \times M$. Aktor mulai dari petak Z, harus mencapai petak keluar O, dan bergerak hanya ke empat arah: atas, bawah, kiri, dan kanan. Perbedaannya dengan pencarian jalur biasa adalah permukaan es: ketika plyaer memilih satu arah, aktor akan terus meluncur sampai berhenti tepat sebelum batu atau dinding X. Jika aktor keluar papan tanpa menabrak batu, maka game over.

Papan juga dapat memuat tile lava atau dilambangkan L dan checkpoint bernomor 0 sampai 9. Jika aktor melewati atau menginjak tile lava, maka game over juga. Checkpoint harus dilewati secara berurutan; misalnya checkpoint 1 baru boleh dilewati setelah checkpoint 0 selesai. Setelah checkpoint yang benar dilewati, petak tersebut dianggap sebagai petak biasa untuk kunjungan berikutnya, jika misal aktor melewati petak 2 tanpa melewati petak 1 dulu, maka game over juga.

Setiap petak memiliki biaya traversal. Biaya satu gerakan adalah jumlah biaya semua petak yang dilewati selama sliding, tidak termasuk petak awal gerakan, tetapi termasuk petak tempat aktor berhenti. Karena itu, tujuan program bukan hanya menemukan jalur, tetapi juga menghitung total biaya jalur tersebut.



Gambar 1.1: GUI: tampilan hasil solve untuk `sample_valid.txt`

1.2 Ruang Lingkup Implementasi

Program dibuat dalam bahasa Java dengan struktur berbasis Maven. Fitur utama yang diimplementasikan adalah:

- pembacaan dan validasi file input `.txt`;
- penyelesaian puzzle menggunakan UCS, Greedy Best First Search, dan A*;
- tiga pilihan heuristik untuk GBFS dan A*;
- bonus GUI JavaFX untuk memilih file, memilih algoritma, menjalankan solver, dan playback solusi.

BAB 2

Algoritma dan Implementasi

2.1 Representasi Papan dan State

Papan direpresentasikan oleh kelas `Board`, sedangkan keseluruhan puzzle disimpan dalam kelas `Puzzle`. Informasi yang disimpan meliputi:

- ukuran papan;
- karakter petak;
- matriks biaya traversal;
- posisi awal;
- posisi tujuan;
- daftar posisi checkpoint berurutan.

State pencarian tidak cukup hanya menyimpan posisi aktor. Program juga menyimpan `nextCheckpointIndex`, yaitu indeks checkpoint berikutnya yang harus dilewati. Dengan begitu, dua state pada posisi aktor yang sama tetap dapat dibedakan apabila progres checkpoint-nya berbeda.

Contoh: State (`posisi=(5,1)`, `nextCheckpointIndex=0`) berarti belum ada checkpoint yang dilewati. State (`posisi=(5,1)`, `nextCheckpointIndex=2`) berarti checkpoint 0 dan 1 sudah dilewati.

2.2 Simulasi Sliding

Setiap ekspansi node menghasilkan calon gerakan ke arah U, D, L, dan R. Untuk setiap arah, simulator melangkah satu petak demi satu petak sampai menemukan X. Jika ditemukan X, aktor berhenti pada petak sebelumnya. Jika keluar papan atau melewati lava, gerakan langsung dianggap tidak valid.

Selama sliding, program juga:

- menjumlahkan biaya petak yang dilewati;
- mengecek checkpoint yang dilewati;
- menolak gerakan jika checkpoint dilewati di luar urutan;

- menyimpan daftar posisi yang dilewati agar dapat divisualisasikan.

Pada setiap algoritma pathfinding, alur umum pencarian dimulai dari state awal, yaitu posisi aktor pada tile Z dengan indeks checkpoint berikutnya bernilai 0. Setiap state yang dipilih dari priority queue kemudian diekspansi dengan mencoba empat arah gerakan. Untuk setiap arah, program menjalankan simulasi sliding sampai aktor berhenti sebelum obstacle, keluar papan, melewati lava, atau melanggar urutan checkpoint. Gerakan yang valid menghasilkan state baru, biaya baru, dan riwayat posisi yang dilewati. State baru hanya dimasukkan ke frontier jika biayanya lebih baik daripada biaya terbaik yang pernah ditemukan untuk state tersebut. Pencarian berhenti ketika aktor berhenti tepat di goal dan seluruh checkpoint sudah dilewati secara berurutan.

2.3 Uniform Cost Search

Fungsi evaluasi UCS adalah:

$$f(n) = g(n)$$

di mana $g(n)$ adalah total biaya dari state awal sampai node n .

UCS menggunakan PriorityQueue yang diurutkan berdasarkan biaya $g(n)$ terkecil. Jika terdapat nilai yang sama, tie-break menggunakan urutan insertion. Setiap kali sebuah node diambil dari antrian, node tersebut hanya diekspansi apabila nilai g -nya sama dengan nilai terbaik yang tersimpan di `bestG`.

```
1 function solveUCS(puzzle):
2     startState = (startPosition, nextCheckpointIndex = 0)
3     push startNode with g = 0, f = 0
4     bestG[startState] = 0
5
6     while priorityQueue not empty:
7         current = pop node with smallest g
8         if current.g != bestG[current.state]:
9             continue
10
11         expand current
12         if current is goal:
13             return solution
14
15         for direction in [U, D, L, R]:
16             slide = simulateSlide(current.state, direction)
```

```
17         if slide invalid:
18             continue
19
20         newG = current.g + slide.cost
21         if newG < bestG[slide.state]:
22             update bestG
23             push successor
```

2.4 Greedy Best First Search

Greedy Best First Search atau GBFS menggunakan heuristik sebagai dasar utama pemilihan node. Fungsi evaluasinya adalah:

$$f(n) = h(n)$$

di mana $h(n)$ adalah estimasi biaya tersisa dari node n ke node tujuan berdasarkan heuristik yang dipilih. GBFS cenderung cepat menuju state yang tampak dekat dengan target, tetapi tidak mempertimbangkan biaya yang sudah dibayar sebelum sampai ke node n . Karena itu, GBFS tidak menjamin solusi dengan biaya minimum. Pada implementasi ini, PriorityQueue GBFS diurutkan berdasarkan $h(n)$, kemudian $g(n)$ lebih kecil, lalu insertion order.

```
1 function solveGBFS(puzzle, heuristic):
2     startState = (startPosition, nextCheckpointIndex = 0)
3     startH = heuristic(startState)
4     push startNode with g = 0, h = startH, f = startH
5     bestG[startState] = 0
6
7     while priorityQueue not empty:
8         current = pop node with smallest h
9         if current.g != bestG[current.state]:
10             continue
11
12         expand current
13         if current is goal:
14             return solution
15
16         for direction in [U, D, L, R]:
17             slide = simulateSlide(current.state, direction)
18             if slide invalid:
```

```

19         continue
20
21     newG = current.g + slide.cost
22     if newG < bestG[slide.state]:
23         newH = heuristic(slide.state)
24         update bestG
25         push successor with g = newG, h = newH, f =
            newH

```

2.5 A* Search

A* menggabungkan biaya aktual yang sudah ditempuh dan estimasi biaya tersisa:

$$f(n) = g(n) + h(n)$$

Nilai $g(n)$ adalah akumulasi biaya gerakan, sedangkan $h(n)$ adalah hasil heuristik. Pada implementasi ini, PriorityQueue A* diurutkan berdasarkan $f(n)$, lalu $h(n)$ lebih kecil, lalu $g(n)$ lebih kecil, lalu insertion order. A* dijamin optimal apabila heuristik yang digunakan admissible dan penanganan state berulang dilakukan dengan benar.

```

1 function solveAStar(puzzle, heuristic):
2     startState = (startPosition, nextCheckpointIndex = 0)
3     startH = heuristic(startState)
4     push startNode with g = 0, h = startH, f = startH
5     bestG[startState] = 0
6
7     while priorityQueue not empty:
8         current = pop node with smallest f
9         if current.g != bestG[current.state]:
10             continue
11
12         expand current
13         if current is goal:
14             return solution
15
16         for direction in [U, D, L, R]:
17             slide = simulateSlide(current.state, direction)
18             if slide invalid:
19                 continue
20

```



```
21         newG = current.g + slide.cost
22         if newG < bestG[slide.state]:
23             newH = heuristic(slide.state)
24             newF = newG + newH
25             update bestG
26         push successor with g = newG, h = newH, f =
            newF
```

2.6 Perbandingan Algoritma

Ketiga algoritma yang digunakan memiliki cara memilih node yang berbeda. Perbedaan ini cukup berpengaruh pada Ice Sliding Puzzle karena satu gerakan tidak hanya berpindah satu petak, tetapi dapat melewati banyak tile sekaligus. Selain itu, setiap tile memiliki traversal cost, checkpoint harus dilewati berurutan, dan beberapa lintasan dapat langsung menjadi invalid jika melewati lava atau keluar papan.

Secara teoritis, UCS adalah pilihan paling aman ketika tujuan utama adalah menemukan solusi dengan total cost minimum. Hal ini karena UCS selalu memproses state dengan $g(n)$ terkecil terlebih dahulu. Pada puzzle berbobot, strategi ini penting karena jalur dengan jumlah gerakan lebih sedikit belum tentu lebih murah; satu slide pendek dapat saja melewati tile mahal, sedangkan slide yang terlihat memutar dapat memiliki cost total lebih rendah.

Lalu untuk GBFS, algoritma ini lebih agresif mengikuti heuristik. Pada Ice Sliding Puzzle, GBFS dapat terlihat cepat ketika heuristik sejalan dengan struktur papan, misalnya ketika checkpoint berikutnya memang berada di arah yang mudah dicapai. Namun, karena GBFS mengabaikan akumulasi cost sebagai prioritas utama, algoritma ini dapat berhenti pada solusi valid pertama yang belum tentu murah. Kasus seperti goal yang tampak dekat tetapi belum boleh dituju karena checkpoint belum selesai menunjukkan kelemahan GBFS dengan heuristik yang terlalu sederhana.

Tabel 2.1: Perbandingan UCS, GBFS, dan A* pada Ice Sliding Puzzle

Algoritma	Fungsi evaluasi	Kelebihan	Kekurangan
UCS	$f(n) = g(n)$	Menjamin solusi dengan cost minimum selama seluruh cost positif. Cocok dijadikan baseline karena keputusan ekspansi sepenuhnya berdasarkan biaya aktual yang sudah ditempuh.	Dapat mengeksplorasi banyak state karena tidak memiliki informasi arah menuju goal atau checkpoint berikutnya. Pada papan besar, jumlah iterasi dapat meningkat.
GBFS	$f(n) = h(n)$	Biasanya lebih cepat mengarah ke target karena memilih state yang terlihat paling dekat menurut heuristik. Berguna untuk melihat pengaruh heuristik terhadap arah pencarian.	Tidak mempertimbangkan $g(n)$ sebagai prioritas utama, sehingga solusi pertama yang ditemukan tidak dijamin memiliki cost minimum. Mudah tertipu oleh goal yang terlihat dekat tetapi jalurnya mahal atau belum memenuhi checkpoint.
A*	$f(n) = g(n) + h(n)$	Menyeimbangkan biaya aktual dan estimasi sisa biaya. Pada banyak kasus, A* lebih terarah daripada UCS tetapi tetap lebih sadar cost daripada GBFS.	Jaminan optimalitas bergantung pada admissibility heuristik. Pada Ice Sliding Puzzle, analisis heuristik lebih sulit karena efek sliding dan checkpoint yang bisa dilewati di tengah gerakan.

A* menjadi kompromi antara keduanya. Nilai $g(n)$ menjaga agar biaya aktual tetap diperhitungkan, sedangkan $h(n)$ membantu pencarian lebih terarah. Dalam konteks tugas ini, hasil A* tetap dibandingkan dengan UCS sebagai baseline optimal. Jika A* menghasilkan cost yang sama dengan UCS, maka pada test case tersebut heuristik membantu mengarahkan pencarian tanpa merusak kualitas solusi. Namun, karena mekanik sliding membuat pembuktian admissibility tidak selalu sederhana, A* tidak

diklaim optimal untuk semua heuristik secara mutlak.

2.7 Heuristik

Program menyediakan tiga heuristik. Seluruh heuristik dikalikan dengan `minPassableTileCost`, yaitu biaya traversal terkecil dari petak yang dapat dilewati. Perkalian ini membuat estimasi berbasis jarak Manhattan berada dalam satuan biaya, bukan sekadar satuan jumlah petak. Pada rumus di bawah, $D(a, b)$ menyatakan jarak Manhattan dari posisi a ke posisi b , dan $C_{\min}$ menyatakan nilai `minPassableTileCost`.

Jarak Manhattan adalah jarak pada grid yang dihitung dari selisih baris dan selisih kolom dua posisi. Jika posisi a berada pada (r_a, c_a) dan posisi b berada pada (r_b, c_b) , maka:

$$D(a, b) = |r_a - r_b| + |c_a - c_b|$$

Jarak ini cocok sebagai estimasi sederhana karena aktor hanya dapat bergerak ke empat arah utama: atas, bawah, kiri, dan kanan. Perlu dicatat bahwa nilai ini belum memperhitungkan obstacle, lava, maupun efek sliding; ia hanya memberikan gambaran kasar kedekatan dua posisi pada papan.

Secara umum, heuristik di program ini tidak mencoba mensimulasikan sliding secara penuh. Tujuannya adalah memberi prioritas arah pencarian dengan perhitungan yang ringan, sehingga algoritma GBFS dan A* tidak harus memperlakukan semua state seolah sama menjanjikannya. Dengan kata lain, heuristik bekerja sebagai kompas kasar yang mana tidak menggantikan aturan validasi gerakan, tetapi membantu menentukan state mana yang lebih masuk akal untuk diproses lebih dahulu.

2.7.1 H1: Manhattan ke Goal

$$h_1(n) = D(\text{current}, \text{goal}) \times C_{\min}$$

H1 menghitung estimasi dari posisi aktor saat ini langsung ke goal. Heuristik ini adalah baseline yang paling sederhana karena hanya membutuhkan dua posisi: posisi sekarang dan posisi goal. Kelebihannya adalah perhitungannya cepat, mudah dipahami, dan cocok untuk melihat bagaimana perilaku pencarian jika informasi yang diberikan ke algoritma hanya berupa “seberapa dekat aktor dengan pintu keluar”.

Dalam implementasi, H1 berguna untuk kasus papan tanpa checkpoint atau papan yang checkpoint-nya secara kebetulan berada di jalur menuju goal. Namun, pada puzzle yang memiliki checkpoint wajib, H1 tidak mengetahui bahwa aktor harus melewati angka

secara berurutan terlebih dahulu. Akibatnya, H1 bisa saja menganggap state yang dekat dengan goal sebagai state yang menarik, walaupun state tersebut belum memenuhi urutan checkpoint. Karena alasan ini, H1 lebih tepat dipakai sebagai pembanding sederhana dibanding sebagai heuristik utama untuk semua jenis papan.

2.7.2 H2: Manhattan ke Target Wajib Berikutnya

$$h_2(n) = D(\text{current}, \text{nextMandatoryTarget}) \times C_{\min}$$

Target wajib berikutnya adalah checkpoint berikutnya jika masih ada, atau goal jika semua checkpoint sudah selesai. Jadi, ketika aktor masih harus melewati checkpoint 0, H2 akan mengarahkan estimasi ke checkpoint 0. Setelah checkpoint 0 berhasil dilewati, targetnya bergeser ke checkpoint 1, dan seterusnya sampai akhirnya target menjadi goal.

Heuristik ini lebih selaras dengan aturan permainan karena memperhatikan state `nextCheckpointIndex`. State yang secara posisi dekat dengan goal tidak otomatis dianggap paling menarik jika checkpoint wajib berikutnya belum dilewati. Dengan cara ini, H2 memberi tekanan pada pencarian agar menyelesaikan tujuan kecil secara berurutan: capai checkpoint berikutnya, lalu checkpoint selanjutnya, baru kemudian goal.

Walaupun masih memakai jarak Manhattan biasa dan belum memperhitungkan letak obstacle secara detail, H2 biasanya lebih stabil dibanding H1 pada papan yang memiliki checkpoint. Biaya komputasinya juga tetap rendah karena setiap pemanggilan heuristik hanya menghitung jarak ke satu target.

2.7.3 H3: Rantai Manhattan Checkpoint

$$h_3(n) = \left(D(\text{current}, c_k) + D(c_k, c_{k+1}) + \dots + D(c_{\text{last}}, \text{goal}) \right) \times C_{\min}$$

H3 memperluas ide H2 dengan tidak hanya melihat target wajib berikutnya, tetapi juga seluruh rangkaian checkpoint yang masih tersisa. Jika checkpoint berikutnya adalah c_k , maka H3 menjumlahkan jarak dari posisi sekarang ke c_k , dari c_k ke c_{k+1} , dan berlanjut sampai checkpoint terakhir menuju goal. Dengan begitu, H3 mencoba memberi gambaran kasar mengenai “sisa perjalanan terstruktur” yang masih harus diselesaikan oleh aktor.

Heuristik ini biasanya lebih informatif daripada H1 dan H2 karena membawa lebih

banyak konteks tentang urutan checkpoint. Dua state yang sama-sama dekat dengan target berikutnya dapat memperoleh nilai berbeda jika setelah target tersebut masih ada rangkaian checkpoint yang panjang. Informasi tambahan ini dapat membantu pencarian memilih cabang yang terlihat lebih menjanjikan untuk keseluruhan puzzle, bukan hanya untuk target paling dekat.

Konsekuensinya, H3 sedikit lebih mahal dihitung karena harus menjumlahkan beberapa jarak Manhattan sekaligus. Namun, karena jumlah checkpoint dibatasi oleh digit 0 sampai 9, biaya tambahan ini tetap kecil dalam praktik. Pada program ini, H3 terutama dipakai untuk eksperimen dan perbandingan performa: apakah heuristik yang lebih banyak informasi dapat mengurangi iterasi, dan bagaimana pengaruhnya terhadap biaya solusi yang ditemukan.

2.8 Admissibility Heuristik

Secara formal, heuristik disebut *admissible* apabila tidak pernah melebihi biaya minimum sebenarnya menuju solusi. Dengan kata lain, untuk setiap state n , nilai $h(n)$ harus lebih kecil atau sama dengan biaya termurah yang masih dibutuhkan dari state tersebut sampai goal.

Pada graf biasa dengan gerakan satu langkah per sisi, jarak Manhattan yang dikalikan biaya minimum sering menjadi kandidat heuristik yang cukup natural. Namun, persoalan Ice Sliding Puzzle punya karakter yang berbeda. Satu aksi tidak selalu berarti berpindah satu petak; aktor akan terus meluncur sampai berhenti tepat sebelum obstacle. Ada posisi yang secara Manhattan dekat dengan target, tetapi sulit dijangkau karena tidak ada obstacle yang membuat aktor berhenti di lokasi yang diperlukan. Sebaliknya, ada juga target yang tampak jauh secara Manhattan, tetapi bisa dilewati dalam satu gerakan sliding panjang.

Selain itu, checkpoint pada puzzle ini tidak harus menjadi titik berhenti. Checkpoint dianggap berhasil jika aktor melewatinya dalam urutan yang benar, meskipun aktor terus meluncur setelah itu. Jarak ke checkpoint bukan selalu berarti biaya untuk berhenti di checkpoint, melainkan hanya estimasi untuk mencapai atau melewati area checkpoint tersebut.

H1

H1 menghitung jarak Manhattan dari posisi aktor ke goal. Untuk kasus tanpa checkpoint, H1 dapat menjadi estimasi sederhana terhadap kedekatan aktor dengan tujuan akhir. Karena dikalikan dengan `minPassableTileCost`, nilainya tidak lebih kecil dari satuan jarak saja, tetapi sudah berada dalam satuan biaya minimum per petak.

Masalah utama H1 muncul ketika masih ada checkpoint yang belum dilewati. Pada kondisi tersebut, goal belum boleh dianggap sebagai tujuan langsung karena solusi baru sah setelah semua checkpoint diselesaikan. H1 bisa memberikan nilai kecil pada state yang dekat dengan goal, padahal state tersebut masih jauh dari checkpoint berikutnya. Oleh karena itu, H1 tidak diperlakukan sebagai heuristik yang kuat untuk seluruh variasi papan; H1 lebih cocok sebagai baseline untuk melihat dampak heuristik paling sederhana.

H2

H2 menaksir jarak menuju target wajib berikutnya, yaitu checkpoint berikutnya jika masih ada, atau goal jika semua checkpoint sudah dilewati. Secara konsep, H2 lebih dekat dengan struktur masalah karena puzzle memang tidak dapat selesai tanpa memenuhi target wajib berikutnya terlebih dahulu. Dibanding H1, H2 tidak terlalu mudah terkecoh oleh goal ketika urutan checkpoint masih belum selesai.

Dari sisi admissibility, H2 adalah pilihan yang lebih aman secara praktis dibanding H1 dan H3 karena ia hanya melihat satu target wajib terdekat dalam urutan penyelesaian. Akan tetapi, saya tetap tidak menyatakan H2 sebagai admissible untuk semua konfigurasi secara mutlak. Alasannya, mekanik sliding dapat membuat biaya aktual menuju atau melewati target berbeda dari estimasi Manhattan. Posisi berhenti, susunan obstacle, dan kemungkinan melewati target di tengah slide dapat membuat pembuktian formalnya tidak sesederhana puzzle grid biasa.

H3

H3 menjumlahkan estimasi jarak dari posisi sekarang ke checkpoint berikutnya, lalu dari checkpoint tersebut ke checkpoint selanjutnya, sampai akhirnya ke goal. Heuristik ini memberi informasi paling banyak karena memperhitungkan urutan target yang masih tersisa. Dalam banyak kasus eksperimen, informasi tambahan seperti ini dapat membantu pencarian mengurangi jumlah state yang diproses.

Walaupun begitu, H3 juga tidak bisa langsung diklaim admissible. Rantai Manhattan menganggap perjalanan dari satu checkpoint ke checkpoint lain sebagai segmen-segmen yang terpisah. Pada Ice Sliding Puzzle, aktor bisa saja melewati lebih dari satu checkpoint dalam satu gerakan jika urutan checkpoint benar dan lintasannya memungkinkan. Karena checkpoint tidak wajib menjadi posisi berhenti, penjumlahan jarak antar checkpoint dapat menjadi estimasi yang terlalu konservatif atau tidak sesuai dengan biaya aktual pada kasus tertentu. Maka, H3 digunakan sebagai heuristik pembanding yang lebih informatif, bukan sebagai heuristik yang dijamin menjaga optimalitas A^* pada semua papan.

Dengan pertimbangan tersebut, implementasi A* pada program ini benar dari sisi mekanisme pencarian: menggunakan $g(n)$ sebagai biaya aktual, $h(n)$ sebagai estimasi, serta **bestG** untuk mencegah ekspansi state yang lebih buruk. Namun, jaminan optimalitas A* tetap bergantung pada sifat heuristik yang dipilih. Karena UCS tidak menggunakan heuristik dan selalu memperluas state berdasarkan biaya aktual terkecil, hasil UCS dipakai sebagai baseline biaya optimal pada eksperimen. Jika A* menghasilkan biaya yang sama dengan UCS, maka pada test case tersebut heuristik berhasil membantu pencarian tanpa mengubah

2.9 Bonus GUI JavaFX

Bonus GUI dibuat menggunakan JavaFX dan Maven. GUI memanggil kelas parser, model, heuristik, dan solver yang sama dengan CLI. Dengan demikian, tidak ada duplikasi logika pencarian pada GUI.

Fitur GUI meliputi:

- memilih file input **.txt**;
- memilih algoritma UCS, GBFS, atau A*;
- memilih heuristik H1, H2, atau H3 untuk GBFS dan A*;
- menampilkan board pada **Canvas**;
- menampilkan solusi, biaya, iterasi, dan waktu eksekusi;
- playback solusi setelah pencarian selesai;
- menyimpan solusi dan snapshot iterasi ke file **.txt**.

Sesuai spesifikasi, playback bukan live update. Pencarian dijalankan terlebih dahulu sampai selesai, kemudian hasilnya divisualisasikan melalui step solusi dan data snapshot iterasi yang sudah disimpan dalam **SearchResult**.

BAB 3

Struktur Program dan Cara Menjalankan

3.1 Struktur Direktori

```
Tucil3_13524014/
|-- pom.xml           # Konfigurasi Maven, Java 21, JavaFX, dan plugin run
|-- bin/             # Folder keluaran executable sesuai struktur tugas
|-- doc/             # Laporan
|-- src/             # Folder source code
|   |-- main/java/    # Source code aktif yang dikompilasi Maven
```

Isi utama src/main/java adalah sebagai berikut:

```
src/main/java/
|-- Main.java         # Entry point CLI
|-- cli/
|   '-- PlaybackController.java # Kontrol playback terminal: next, previous,
      jump
|-- gui/
|   |-- BoardView.java      # Canvas renderer untuk visualisasi board JavaFX
|   |-- GuiMain.java        # Launcher JavaFX
|   |-- IceSlidingApp.java   # Window utama GUI, event handler, dan integrasi
      solver
|   '-- StepPlaybackModel.java # Model indeks step untuk playback GUI
|-- model/
|   |-- Board.java          # Representasi grid tile, cost, dan utilitas
      board
|   |-- Direction.java      # Enum arah gerak: Up, Down, Left, Right
|   |-- Position.java       # Koordinat baris dan kolom
|   |-- Puzzle.java         # Gabungan board, start, goal, dan checkpoint
|   '-- TileType.java       # Klasifikasi karakter tile pada input
|-- parser/
|   |-- InvalidPuzzleException.java # Exception untuk input puzzle yang tidak valid
|   '-- PuzzleParser.java      # Pembaca file .txt dan validator format puzzle
|-- solver/
|   |-- AStarSolver.java     # Implementasi A* dengan  $f(n)=g(n)+h(n)$ 
|   |-- GreedyBestFirstSolver.java # Implementasi GBFS dengan  $f(n)=h(n)$ 
|   |-- Heuristic.java      # Interface fungsi heuristik
|   |-- Heuristics.java     # Factory dan implementasi H1, H2, H3
|   |-- HeuristicType.java   # Enum pilihan heuristik
|   |-- SearchAlgorithm.java # Enum pilihan algoritma pencarian
|   |-- SearchNode.java      # Node pencarian: parent, move, g, h, f
|   |-- SearchResult.java    # Hasil search: solusi, cost, iterasi, waktu,
      step
|   |-- SearchState.java     # Identitas state: posisi aktor dan checkpoint
      berikutnya
|   |-- SlideResult.java     # Hasil simulasi satu gerakan sliding
|   |-- SlideSimulator.java  # Validasi dan simulasi gerakan di atas es
|   '-- UniformCostSolver.java # Implementasi UCS dengan  $f(n)=g(n)$ 
'-- util/
```



```
|-- BoardRenderer.java      # Renderer board untuk output teks CLI/file
|-- PathReconstructor.java  # Rekonstruksi path dari final node ke initial
    state
'-- SolutionWriter.java     # Penyimpanan solusi dan snapshot iterasi ke .
    txt
```

Folder `src/gui` terlihat sebagai folder GUI kedua di direktori lokal, tetapi folder tersebut kosong dan tidak masuk jalur kompilasi Maven. GUI yang benar-benar digunakan berada pada package `gui` di `src/main/java/gui`. Dengan layout ini, CLI dan GUI tetap berbagi parser, model, simulator, heuristik, dan solver yang sama.

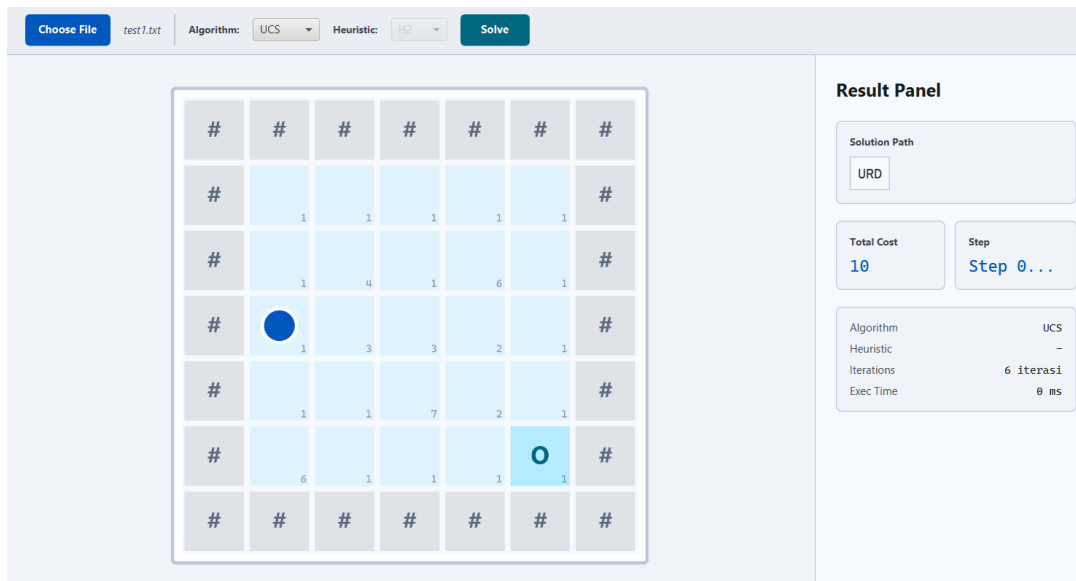
3.2 Cara Menjalankan

```
Cara Kompilasi
mvn clean compile
Cara menjalankan CLI
mvn exec:java
Cara Menjalankan GUI
mvn javafx:run
```

BAB 4

Eksperimen dan Hasil Pengujian

4.1 Test Case 1

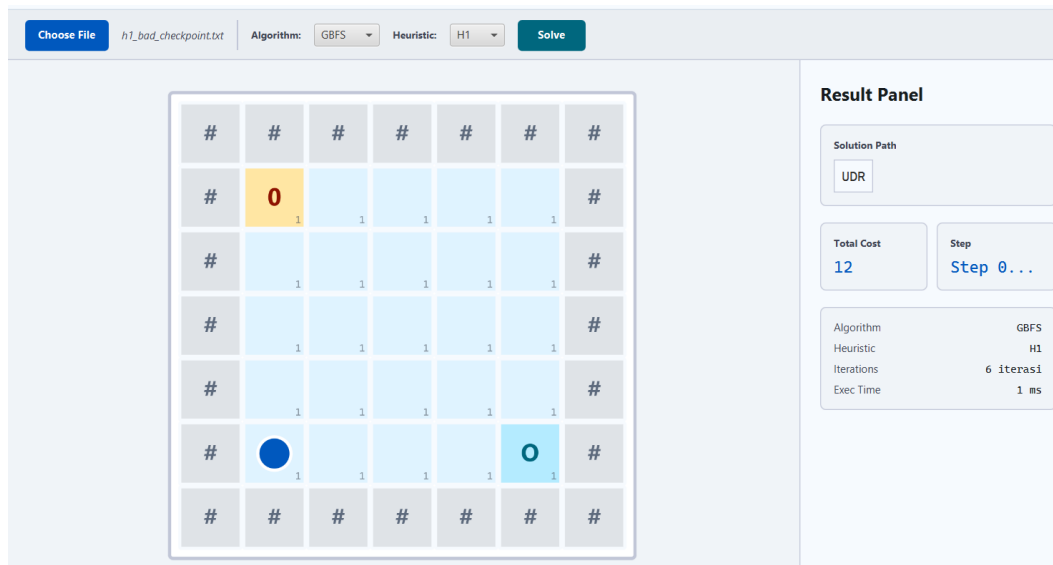


Gambar 4.1: Input dan Output Test Case 1

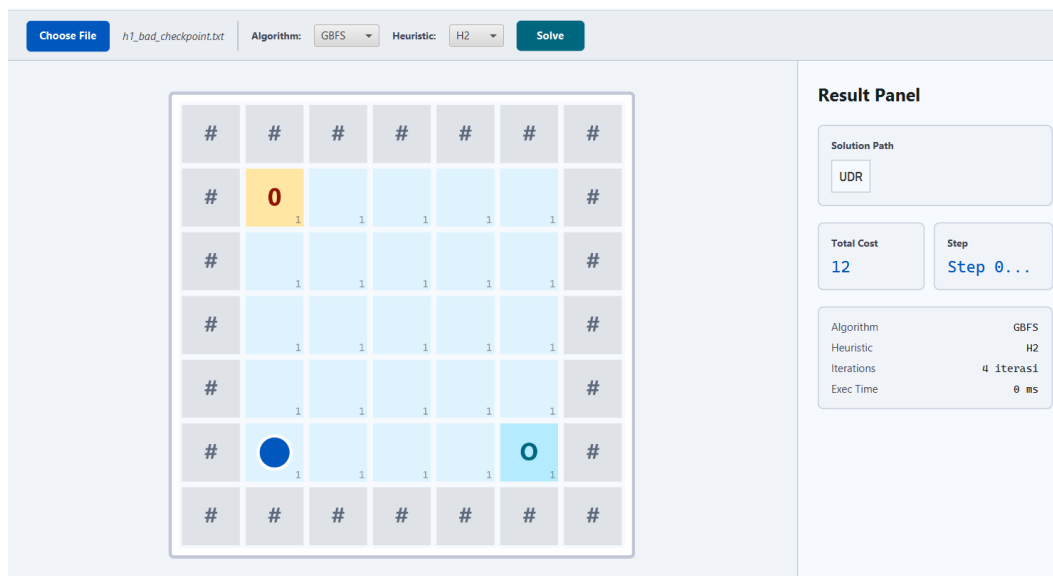
Analisis: UCS menghasilkan solusi URD dengan total cost 10 dan 6 iterasi. Karena tidak terdapat checkpoint, tujuan pencarian hanya mencapai petak goal 0 dengan biaya minimum. Jalur yang dipilih UCS tidak sekadar mencari jumlah gerakan paling sedikit, tetapi mempertimbangkan akumulasi traversal cost dari setiap petak yang dilewati saat sliding.

Solusi URD menunjukkan bahwa UCS berhasil menghindari jalur yang terlihat langsung lebih pendek secara jarak kotak tetapi memiliki biaya lebih besar. Total cost 10 diperoleh dari penjumlahan biaya tile yang dilalui pada tiga gerakan tersebut. Hasil ini memperlihatkan sifat UCS yang optimal terhadap bobot biaya: selama semua biaya positif, state dengan cost terkecil akan diproses lebih dahulu sehingga solusi pertama yang mencapai goal merupakan solusi dengan biaya minimum

4.2 Test Case 2



Gambar 4.2: Input dan Output Test Case 2 dengan GBFS H1



Gambar 4.3: Input dan Output Test Case 2 dengan GBFS H2

Analisis: Test case 2 menunjukkan kelemahan H1 pada puzzle yang memiliki checkpoint. Secara posisi, goal O berada dekat dengan start Z, sehingga H1 cenderung menilai state yang mendekati goal sebagai lebih menjanjikan. Padahal, solusi belum sah sebelum checkpoint 0 dilewati terlebih dahulu.

Akibatnya, GBFS dengan H1 melakukan lebih banyak eksplorasi karena heuristic tidak mengetahui bahwa target wajib pertama sebenarnya adalah checkpoint, bukan

goal. Pada kasus ini, GBFS-H1 membutuhkan 6 iterasi, sedangkan GBFS-H2 hanya membutuhkan 4 iterasi. H2 lebih efisien karena langsung mengarahkan pencarian ke checkpoint berikutnya sebelum mempertimbangkan goal.

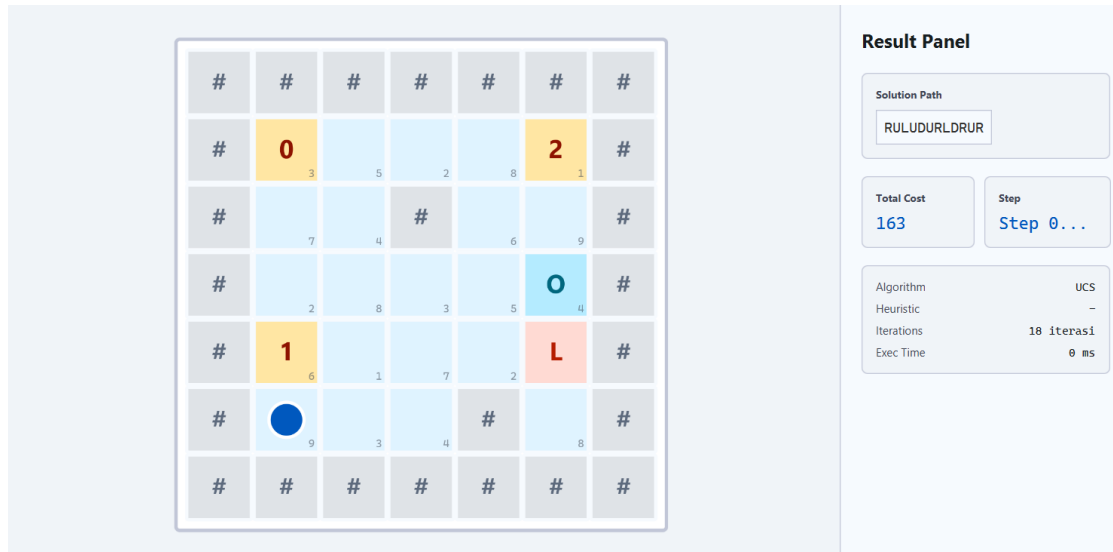
Walaupun solusi optimal pada test case ini memiliki cost 10, GBFS dengan H1 menghasilkan solusi berbiaya 12. Hal ini terjadi karena GBFS hanya menggunakan nilai heuristic sebagai dasar prioritas, yaitu $f(n) = h(n)$, dan tidak menjadikan biaya yang sudah dikeluarkan $g(n)$ sebagai komponen utama evaluasi. Dengan H1, state yang terlihat lebih dekat ke goal dapat dipilih lebih dahulu meskipun jalur menuju state tersebut bukan jalur termurah.

Pada puzzle ini, checkpoint 0 wajib dilewati sebelum goal, tetapi H1 tetap menghitung estimasi langsung ke goal. Akibatnya, pencarian terdorong memilih urutan gerakan yang cepat terlihat mendekati goal setelah checkpoint dilewati, bukan urutan gerakan dengan akumulasi cost minimum. Karena GBFS berhenti ketika menemukan goal valid pertama, solusi yang ditemukan belum tentu merupakan solusi dengan cost paling kecil. Inilah sebabnya solusi GBFS-H1 berhenti pada cost 12, sedangkan cost optimal sebenarnya adalah 10.

4.3 Test Case 3



Gambar 4.4: Input dan Output Test Case 3 dengan A* H3

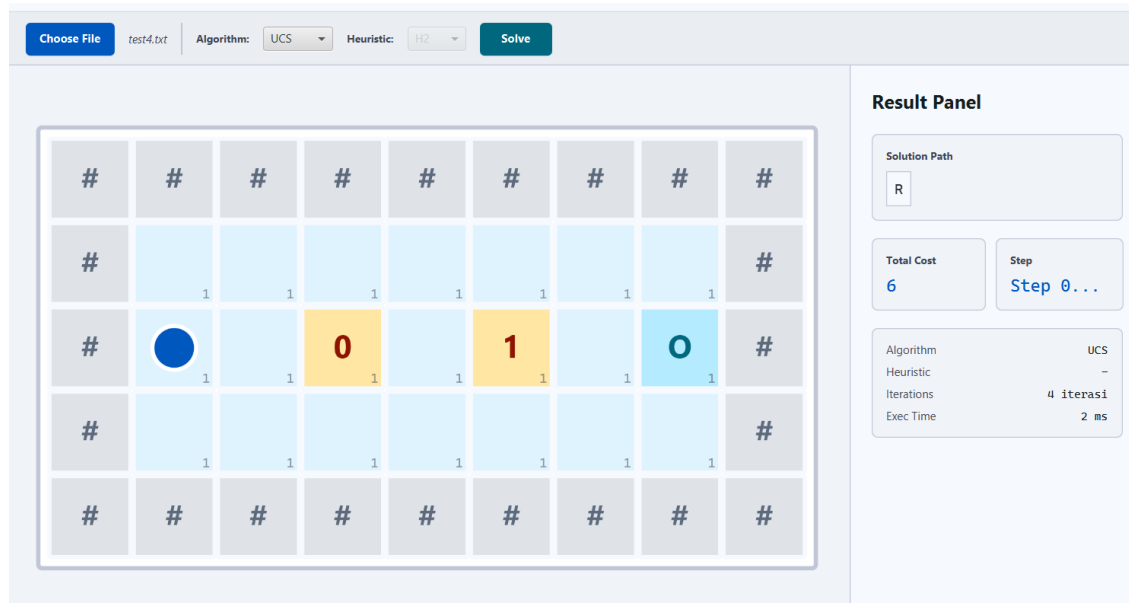


Gambar 4.5: Input dan Output Test Case 3 dengan UCS

Analisis: Pada pengujian, A* dengan H3 menemukan solusi RULUDURLDRUR dengan total cost 163 dan 18 iterasi. Hasil ini sama dengan UCS pada test case yang sama, sehingga pada kasus ini A* H3 berhasil menemukan solusi dengan cost yang sesuai baseline optimal UCS. Hal ini menunjukkan bahwa walaupun H3 bersifat lebih eksperimental, informasi urutan checkpoint yang dimasukkan ke dalam heuristik tetap dapat mengarahkan pencarian ke solusi valid.

Namun, jumlah iterasi A* H3 pada test case ini tidak lebih kecil dari UCS, yaitu sama-sama 18 iterasi. Hal ini dapat terjadi karena ruang state pada papan relatif terbatas dan jalur valid sangat dipengaruhi oleh posisi obstacle serta lava. Dengan kata lain, H3 membantu merepresentasikan urutan target, tetapi tidak selalu otomatis mengurangi jumlah ekspansi pada setiap papan. Hasil ini tetap berguna untuk menunjukkan bahwa H3 bekerja benar pada kasus multi-checkpoint, sambil tetap tidak diklaim selalu admissible atau selalu lebih cepat dari UCS.

4.4 Test Case 4

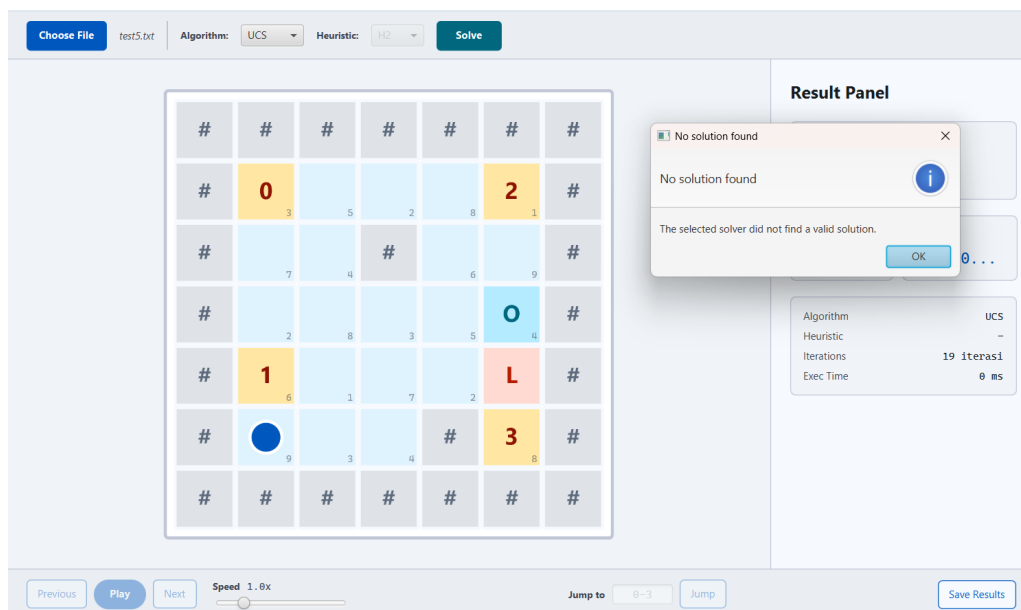


Gambar 4.6: Input dan Output Test Case 4

Analisis: Test case 4 menguji salah satu aturan penting pada Ice Sliding Puzzle, yaitu checkpoint tidak harus menjadi posisi berhenti. Pada papan ini, checkpoint 0 dan 1 berada dalam satu lintasan lurus antara start dan goal. Ketika aktor bergerak ke kanan, aktor meluncur melewati checkpoint 0, lalu checkpoint 1, kemudian berhenti tepat di goal karena ada obstacle di sebelah kanan goal.

UCS menemukan solusi R dengan total cost 6. Cost tersebut berasal dari enam petak yang dilewati selama satu kali slide, termasuk petak checkpoint dan goal, tetapi tidak termasuk petak awal. Test case ini menarik karena solusi valid hanya membutuhkan satu gerakan, namun tetap harus memperbarui `nextCheckpointIndex` dua kali dalam satu slide. Jika implementasi hanya menganggap checkpoint sah ketika aktor berhenti di atasnya, maka test case ini akan gagal.

4.5 Test Case 5



Gambar 4.7: Input dan Output Test Case 5

Analisis: Program tidak menemukan solusi karena urutan checkpoint tidak dapat dipenuhi secara valid. Papan memiliki checkpoint 0, 1, 2, dan 3, sehingga aktor wajib melewati checkpoint tersebut secara berurutan sebelum goal dianggap sah. Dari posisi awal, beberapa lintasan sliding memang dapat mencapai area dekat goal atau checkpoint lain, tetapi lintasan tersebut tidak membentuk urutan 0–1–2–3 yang valid.

Selain itu, terdapat lava L di dekat goal dan checkpoint 3. Karena setiap gerakan sliding harus terus berjalan sampai berhenti sebelum obstacle, aktor tidak bisa bebas berhenti di tengah jalan untuk menghindari tile berbahaya atau memperbaiki urutan checkpoint. Akibatnya, pilihan gerakan yang tersedia selalu berujung pada salah satu kondisi invalid: melewati checkpoint di luar urutan, melewati lava, atau berhenti pada posisi yang tidak membuka akses ke checkpoint berikutnya. Oleh karena itu, test case ini memang tidak memiliki rangkaian gerakan valid dari start menuju goal setelah semua checkpoint dilewati.

BAB 5

Penutup

5.1 Kesimpulan

Program Ice Sliding Puzzle Solver berhasil diimplementasikan dengan tiga algoritma pathfinding: UCS, GBFS, dan A*. Program dapat membaca input `.txt`, memvalidasi papan, mensimulasikan mekanik sliding, menangani lava dan checkpoint berurutan, serta menghitung cost solusi berdasarkan biaya traversal petak yang dilewati.

UCS berperan sebagai pembanding biaya optimal. GBFS dapat mengurangi jumlah ekspansi pada kasus tertentu, tetapi tidak menjamin biaya minimum. A* lebih seimbang karena memakai $g(n)$ dan $h(n)$ sekaligus, tetapi klaim optimalitas tetap bergantung pada sifat heuristic. Pada implementasi ini, H2 menjadi heuristic praktis yang paling masuk akal karena memperhatikan target wajib berikutnya.

Bonus GUI JavaFX juga berhasil dibuat. GUI mempermudah pemilihan file, pemilihan algoritma, visualisasi board, playback solusi, dan penyimpanan hasil. Untuk kebutuhan demo, GUI terasa lebih nyaman dibanding hanya melihat output terminal, terutama pada papan besar.

5.2 Saran

Tugas kali ini cukup menarik, meskipun tidak semenarik tugas STIMA yang kemarin kemarin, mungkin asisten bisa mencari game yang bisa dijadikan spek sehingga tugas lebih menarik, hehe

5.3 Repository

Seluruh implementasi kode program tersedia pada repository berikut:

https://github.com/ucupsss/Tucil3_13524014

5.4 Checklist Pemenuhan Spesifikasi

Tabel 5.1: Checklist kelengkapan program

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas <code>.txt</code> serta menyimpan solusi dalam berkas <code>.txt</code>	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib, yaitu A dan B		✓
7	Ada tambahan dua alternatif heuristik selain yang utama, yaitu H2 dan H3	✓	

5.5 Pernyataan tidak melakukan kecurangan

PERNYATAAN

Kami yang bertanda tangan di bawah ini menyatakan bahwa Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Yusuf Faishal Listyardi